StateGen: State-Based Code Generation with Local Backtracking and Transition Learning for Library-Oriented Programming

Joseph Maina
CMU-AFRICA
jmatheng.andrew.cmu.edu

Patricia Muriira
CMU-Africa
pmuriira@andrew.cmu.edu

Patrick Niyigena
CMU-Africa
patrickn@andrew.cmu.edu

Joshua Wisdom Momo
CMU-Africa
jmomo@andrew.cmu.edu

Samuel Chukwuma
CMU-Africa
schukwum@andrew.cmu.edu

1 Problem Statement

Current LLM code generators waste computational resources by regenerating entire programs when any component fails. For library-oriented tasks (e.g., data science), this is particularly inefficient as solutions naturally decompose into sequential operations (import $\rightarrow$ load $\rightarrow$ transform $\rightarrow$ compute). When state 3 of 5 fails, existing approaches discard all previous work and start over.

We propose **StateGen**, an MDP-based framework that decomposes problems into verifiable states and implements **local backtracking**—retrying only failed states while preserving successful ones. StateGen learns transition patterns across problems for code reuse, achieving 30-40% token reduction while improving accuracy.

Related Work: Self-Planning [2] decomposes into planning/implementation but regenerates all code on failure. Self-Debugging [1] refines complete programs, not granular states. ReAct [4] combines reasoning and acting for general tasks but lacks local backtracking and transition learning. Our contribution: state-level verification with local backtracking and non-stationary policy learning for code generation efficiency.

2 Data

DS-1000 [3]: 1,000 data science problems (NumPy, Pandas, PyTorch, etc.) from StackOverflow. Average 3.6-line solutions with 1.6 test cases. We use 200 problems (100 NumPy + 100 Pandas), split 80/20 for training/evaluation. Downloaded from Hugging Face. *Relevance*: Problems naturally decompose into library operations with clear state boundaries.

BigCodeBench [5]: 1,140 multi-library tasks (139 libraries, 12-20 line solutions). We use 50 moderate-complexity tasks. Downloaded from Hugging Face. *Relevance*: Tests cross-library coordination where lookahead verification prevents cascading failures.

3 Method

3.1 MDP Formulation

We model code generation as a Markov Decision Process (MDP): **States** (s_i) = partial programs; **Actions** = LLM-generated code; **Transition** $T(s_{i+1}|s_i, code) = 1$ if verification passes; **Reward** $R = +1$ (success) or -1 (fail); **Policy** $\pi(code|s_i, ctx)$ = LLM + transition memory.

Key innovation: Non-stationary policy that improves via pattern accumulation.

Connection to Synthesis: Extends CEGIS (Counter-Example Guided Synthesis) with to state-level granularity.The verification errors guide local refinement, not global regeneration.

3.2 Algorithm

1. **Decompose**: LLM splits problem into states $S = \{s_0, ..., s_n\}$ with pre/postconditions
2. **Generate & Verify**: For each s_i:
   - Check memory for pattern $(s_i, ctx) \rightarrow code$. If found, reuse
   - Else: LLM generates code
   - Execute and verify postconditions
   - **Fail**: Retry *only* s_i with error feedback (local backtracking)
   - **Success**: Store pattern, advance to s_{i+1}
3. **Learn**: Memory $M = \{(state, context) \rightarrow code\}$ with semantic retrieval

Extensions: (1) *Lookahead verification* simulates future states to prevent cascading failures (static/type/execution levels).

(2) *Adaptive few-shot* uses retrieved patterns as in-context examples.

Implementation: StateGen uses a multi-agent system with four interacting components:
(1) State Controller orchestrates workflow and manages the feedback loop
(2) LLM Agent (DeepSeek-Coder-V2-16B or any other opensource model) generates code for each state
(3) Execution Engine safely verifies code in sandboxed subprocesses with timeout protection
(4) Memory Manager stores/retrieves patterns using sentence-transformers (cosine similarity >0.85).
The key feedback mechanism: when verification fails at state s, the execution error is fed back to the LLM to retry only s (not s through s), enabling local backtracking.

4 Evaluation

4.1 Metrics

Standard: pass@k (unbiased estimator $1 - n - ck/nk$), success rate.
Novel: (1) Iteration efficiency (state retries vs. full regenerations)
(2) Token efficiency (total tokens/solution)
(3) Transition reuse rate (pattern reuse %)
(4) Lookahead effectiveness (conflicts detected vs. runtime failures).

4.2 Baselines & Success Criteria

Baselines: Direct generation, Self-Planning [2], Self-Debugging [1].

Success: (1) 10-15% pass@1 improvement over direct generation
(2) 25-40% token reduction vs. Self-Planning
(3) >20% transition reuse rate.

References

[1] Xinyun Chen, Maxwell Lin, Nathanael Schärli, and Denny Zhou. Teaching large language models to self-debug. In *International Conference on Learning Representations (ICLR)*, 2024.

[2] Xue Jiang, Yihong Dong, Lecheng Wang, Zheng Fang, Qiwei Shang, Ge Li, Zhi Jin, and Wenpin Jiao. Self-planning code generation with large language models. *ACM Transactions on Software Engineering and Methodology*, 2024. Presented at ASE 2024.

[3] Yuhang Lai, Chengxi Li, Yiming Wang, Tianyi Zhang, Ruiqi Zhong, Luke Zettlemoyer, Scott Wen-tau Yih, Daniel Fried, Sida Wang, and Tao Yu. Ds-1000: A natural and reliable benchmark for data science code generation. In *International Conference on Machine Learning (ICML)*, 2023.

[4] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.

[5] Terry Yue Zhuo, Minh Chien Vu, Jenny Chim, Han Hu, Wenhao Yu, Ratnadira Widyasari, Imam Nur Bani Yusuf, Haolan Zhan, Junda He, Indraneil Paul, et al. Bigcodebench: Benchmarking code generation with diverse function calls and complex instructions. In *International Conference on Learning Representations (ICLR)*, 2025.